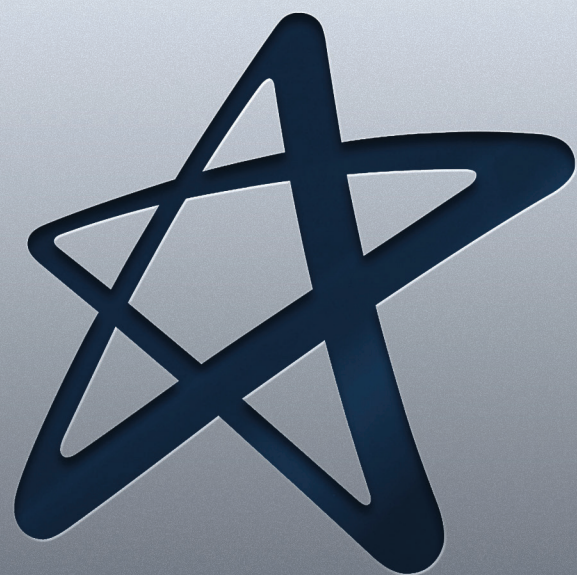


Estruturas de Dados Lineares



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Estruturas de Dados Dinâmicas

Responsável pelo Conteúdo:

Prof. Dr. Luciano Rossi

Revisão Textual:

Prof.^a Esp. Kelciane da Rocha Campos

UNIDADE

Estruturas de Dados Dinâmicas



- Introdução;
- Listas Ligadas;
- Listas Duplamente Ligadas;
- Fila Dinâmica;
- Pilha Dinâmica;
- Algoritmos de Classificação em Estruturas Dinâmicas;
- Pesquisa em Estruturas Dinâmicas.



OBJETIVOS DE APRENDIZADO

- Apresentar ao aluno o conceito de conjuntos de dados dinâmicos, a partir da alocação dinâmica de memória (em tempo de execução);
- Compreender a aplicação de ponteiros em aplicações computacionais, a partir do estudo das estruturas de dados fundamentais implementadas sobre conjuntos dinâmicos;
- Implementar os conceitos de classificação e pesquisa em estruturas dinâmicas.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e sites para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Nesta unidade, trataremos de estruturas de dados que são resultado de alocação dinâmica de memória. Nas unidades anteriores, as estruturas analisadas consideravam alocação estática de memória. As principais diferenças que podemos observar entre as duas formas de alocação de memória são:

- na alocação estática, os *bytes* são alocados de forma contígua (sequencial) na memória; na alocação dinâmica, os *bytes* são distribuídos pela memória;
- na alocação estática, a quantidade de objetos que serão armazenados é conhecida previamente; na alocação dinâmica, não é necessária essa informação prévia;
- a alocação estática é feita em tempo de compilação, enquanto a alocação dinâmica é feita em tempo de execução.

Os objetos que são alocados dinamicamente são compostos pelos valores que queremos armazenar e, pelo menos, um membro adicional que armazenará um endereço de memória. Esse último membro é utilizado para o sequenciamento dos objetos na memória, ou seja, um objeto armazena, além de seus próprios dados, o endereço do próximo objeto. Dessa forma, é possível localizar um objeto a partir de outro objeto.

Vamos iniciar o estudo das estruturas dinâmicas de dados a partir da lista ligada ou encadeada. Essa estrutura tem um objetivo similar ao do vetor (arranjo), que é o armazenamento de uma sequência de elementos. No entanto, o vetor é uma estrutura estática, enquanto a lista ligada é uma estrutura dinâmica. Essa diferença implica mecanismos de funcionamento distintos para cada tipo de estrutura.

Listas Ligadas

A lista ligada é uma estrutura dinâmica de dados cujo objetivo é armazenar elementos de forma sequencial. Nesse contexto, o elemento da lista é armazenado em conjunto com o endereço para o próximo elemento. Esse conjunto é denominado célula. A Figura 1 apresenta um exemplo de lista ligada, cada célula contém o valor do elemento e o endereço (ponteiro) para o próximo elemento da lista. Tanto o valor quanto o ponteiro são denominados de membros da célula (ou estrutura). A última célula contém o valor *Null* no membro referente ao ponteiro, esse valor é representado com a barra na última célula na figura.



Figura 1 – Exemplo de lista ligada

Fonte: Acervo do Conteudista

As listas ligadas pertencem à categoria dos conjuntos dinâmicos, os quais são manipulados por algoritmos e podem aumentar ou diminuir de tamanho e sofrer mudanças

em relação aos elementos que o compõem. Nesse sentido, os conjuntos dinâmicos suportam operações que são categorizadas como operações de consulta ou modificadoras.

As operações de consulta são aquelas que retornam informações a respeito do conjunto, tais como: (i) busca por um determinado elemento, identificar o (ii) menor ou (iii) maior valor no conjunto e retornar o (iv) sucessor ou o (v) predecessor de um determinado elemento.

As operações modificadoras são aquelas que alteram a composição do conjunto. Essas operações se referem à (i) inserção ou (ii) remoção de um determinado elemento do conjunto.

Algoritmo 1 – Algoritmo para busca em uma lista ligada

A operação de busca em uma lista ligada tem suas instruções descritas pelo Algoritmo 1. O algoritmo *BuscaLigada()* recebe o endereço para a lista *L* e o valor *k* a ser procurado na lista. Na linha 1, o endereço da lista é copiado para a variável *x*, assim *x* contém o endereço para a primeira célula. Veja que a notação *x.valor* é o membro da célula que contém o elemento armazenado na lista e *x.ponteiro* é o membro que armazena o endereço da próxima célula. Para o caso de a lista estar vazia, o endereço em *x* não apontará para outra célula (*x = Null*).

A linha 3 inicia um laço que percorrerá a lista, comparando o valor procurado com o valor da célula atual *e*, enquanto não chegar à última célula (*x ≠ Null*), o algoritmo avança para a próxima célula (*x ← x.ponteiro*) em busca do valor procurado. Quando o valor é identificado (*x.valor = k*), o algoritmo retorna o endereço da célula que contém esse valor (*x*). Caso o valor procurado não esteja na lista, o algoritmo retornará *Null*.

BuscaLigada(L, k)

1. $x \leftarrow L$
2. **enquanto** *x ≠ Null* e *x.valor ≠ k* **faça**
3. $x \leftarrow x.ponteiro$
4. **retornar** *x*



A alocação dinâmica de memória é feita de diferentes formas, a depender da linguagem de programação considerada. Veja como a alocação é feita na linguagem C:

```
typedef struct celula {  
    int valor;  
    struct celula* ponteiro;  
}Celula;
```

A operação de inserção em uma lista ligada é feita de maneira simples, primeiro se deve alocar uma nova célula (*nova*). O elemento que será inserido na fila é atribuído ao membro *nova.valor* e o membro *nova.ponteiro* deverá conter o endereço da primeira posição atual da lista. Por fim, o endereço da nova célula será a “cabeça” da lista ligada.

Veja na Figura 2 um exemplo de inserção de um elemento no início de uma lista ligada; os destaques em vermelho na figura representam ponteiros para endereços de memória.

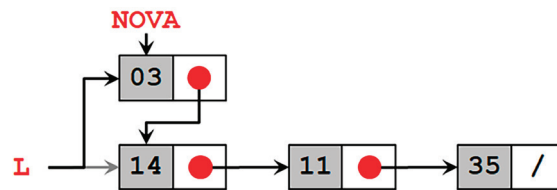


Figura 2 – Inserção no início de uma lista ligada

Fonte: Acervo do Conteudista

Algoritmo 2 – Algoritmo para inserção no início em uma lista ligada

O procedimento *InserirLigada()*, que tem suas instruções descritas no Algoritmo 2, recebe como parâmetros de entrada o endereço da lista ligada (L) e um elemento que deverá ser inserido (k). Inicialmente, é feita a alocação de memória para uma nova célula (linha 1), cujo endereço é armazenado na variável *nova*. Essa nova célula receberá o elemento a ser inserido no membro *nova.valor* e o membro *nova.ponteiro* receberá o endereço do início da lista ligada atual, ou seja, antes da inserção ser realizada (linhas 2 e 3).

A finalização da operação de inserção é feita com a atribuição do endereço da nova célula à variável L , que representa o início da lista ligada.

InserirLigada(L, k)

1. $nova \leftarrow \text{endereço de uma nova célula}$
2. $nova.valor \leftarrow k$
3. $nova.ponteiro \leftarrow L$
4. $L \leftarrow nova$

A remoção de um elemento em uma lista ligada é feita em conjunto com a busca pelo elemento. Quando o elemento é identificado, ele é removido da lista. Nessa operação, devemos manter os endereços da célula atual e da próxima célula. Na Figura 3, as variáveis ponteiro p e q são responsáveis por armazenar esses endereços.



Considerando o ponto de vista da programação, em algumas linguagens a memória alocada deve ser explicitamente liberada pelo programador.

A cada iteração, os ponteiros p e q avançam pela lista, comparando o membro $q.valor$ com o elemento que se deseja remover. Quando o elemento é identificado, o membro $p.ponteiro$ recebe o endereço armazenado no membro $q.ponteiro$, de modo a “desconectar” a célula que contém o elemento procurado. Veja na Figura 3 um exemplo no qual se deseja remover o elemento 11.

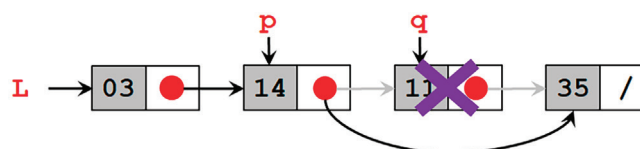


Figura 3 – Busca e remoção em uma lista ligada

Fonte: Acervo do Conteudista

Algoritmo 3 – Algoritmo para remoção em uma lista ligada

Os passos do algoritmo *RemoverLigada()* são descritos no Algoritmo 3. Nesse caso, os parâmetros de entrada são o endereço da lista ligada (L) e o valor que se quer remover (k). O endereço do primeiro elemento na lista é copiado para a variável p (linha 1) e o endereço do segundo elemento, que é descrito pelo membro $L.ponteiro$, é copiado para a variável q (linha 2). Desse modo se tem os endereços de memória do elemento atual e de seu sucessor.

O laço descrito na linha 3 verifica se o ponteiro q excedeu o limite do último elemento da lista e se o valor armazenado no respectivo membro apontado por q corresponde ao elemento buscado. No caso de ambas as verificações não se confirmarem, os ponteiros avançam para as próximas posições na lista (linhas 4 e 5).

Veja que existem duas formas de se encerrar o laço. Caso $q = \text{Null}$, o valor procurado não foi encontrado e nenhuma remoção é realizada; por outro lado, caso $q.valor = k$, isso indica que o elemento buscado foi encontrado. Assim, há a remoção do elemento, por meio da atribuição do endereço do próximo elemento ao membro correspondente no elemento anterior (linha 7).

```

RemoverLigada( $L, k$ )
1.   $p \leftarrow L$ 
2.   $q \leftarrow L.ponteiro$ 
3.  enquanto  $q \neq \text{Null}$  e  $q.valor \neq k$  faça
4.       $p \leftarrow q$ 
5.       $q \leftarrow q.ponteiro$ 
6.  se  $q \neq \text{Null}$ 
7.       $p.ponteiro \leftarrow q.ponteiro$ 
    
```

Os exemplos de operações, anteriormente descritos, podem ser modificados de modo a possibilitar as demais operações. As operações para a identificação do menor e maior valor em uma lista ligada podem ser feitas a partir do Algoritmo 1 (*BuscaLigada()*). Nesse caso, deve-se desconsiderar o elemento procurado e a cada passo na lista, o valor é verificado para se identificar o menor ou maior elemento. Após o percurso por toda lista, deve-se retornar o valor armazenado.

As operações de identificação do sucessor ou predecessor de um determinado elemento podem ser realizadas, também, com base no Algoritmo 1. Veja que a alteração para esse caso é mais simples; para obter o sucessor, basta retornar o endereço armazenado no membro ponteiro do elemento identificado. Para o caso da operação prede-

cessor, devemos manter o endereço do elemento atual e do próximo elemento. Assim, utilizaremos o valor do próximo elemento para a busca pelo elemento objetivo e, quando identificado, retorna o endereço do elemento atual.



Na alocação estática de memória, os elementos em um arranjo estão, de fato, organizados de forma sequencial. Na alocação dinâmica, o sequenciamento é feito por meio dos ponteiros.

A configuração de uma lista ligada, conforme foi descrito anteriormente, considera a representação associada de um membro para o elemento que será incluído na lista e outro membro para o endereço do próximo. Há outra possibilidade, que considera dois membros para armazenamento dos endereços dos elementos anterior e posterior, essa representação é denominada de lista duplamente ligada.

Listas Duplamente Ligadas

As listas duplamente ligadas consideram, para a composição de uma célula, no mínimo, três membros. Um dos membros é utilizado para o armazenamento do elemento na lista, os outros dois são utilizados para armazenar os endereços do elemento anterior e posterior.

A configuração de uma lista duplamente ligada apresenta como vantagem melhor acessibilidade aos elementos na lista. Porém é preciso considerar que será utilizado o dobro de ponteiros e isso acarreta maior quantidade de trabalho para a sua manutenção.

A Figura 4 apresenta um exemplo de lista duplamente ligada; veja que cada célula contém ponteiros para os elementos posterior e anterior, respectivamente. A primeira célula não apresenta ponteiro para o elemento anterior; por motivos óbvios, da mesma forma, a última célula não apresenta ponteiro para o elemento posterior. Dessa forma, a identificação do primeiro e do último elemento na fila pode ser feita a partir dessas características. O primeiro elemento apresenta $L.anterior = Null$ e o último elemento é definido por $L.posterior = Null$.

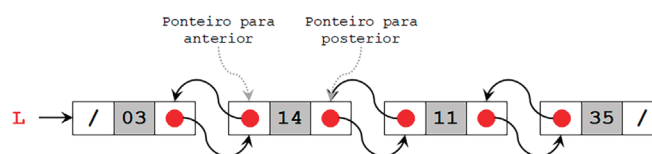


Figura 4 – Exemplo de lista duplamente ligada

Fonte: Acervo do Conteudista

A lista duplamente ligada suporta as mesmas operações que a lista ligada simples. A operação de busca em uma lista duplamente ligada segue de maneira virtualmente idêntica ao descrito no Algoritmo 1, exceto pela denominação do membro, que, neste caso, é denominado “posterior” em vez de “ponteiro”.

A Figura 5 ilustra a operação de inserção em uma lista duplamente ligada. A utilização de uma “cabeça” para a lista, no caso a variável L , possibilita que a inserção seja feita no início da lista. Uma nova célula é alocada para receber o elemento que será inserido.

O próximo passo é a atualização dos ponteiros, o membro “posterior” da nova célula recebe o endereço do primeiro elemento atual. Assim, como a nova célula será a estrutura inicial da lista, o seu membro “anterior” recebe a atribuição do valor *Null*.

Veja que o primeiro elemento atual da fila irá receber, no seu membro “anterior”, o endereço da nova célula e a variável L , que indica a “cabeça” da lista, receberá a atribuição do endereço da nova célula.

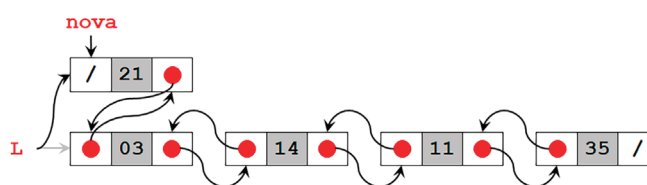


Figura 5 – Operação de inserção em uma lista duplamente ligada
Fonte: Acervo do Conteudista

Algoritmo 4 – Algoritmo para inserção em uma lista duplamente ligada

O Algoritmo 4 descreve as instruções que compõem o procedimento *InserirDLigada()*. Uma nova célula é alocada na memória (linha 1) e o seu endereço é atribuído à variável *nova*. A seguir, na linha 2, o elemento a ser inserido na lista é atribuído ao membro “valor” da nova célula e o seu membro “posterior” recebe o endereço apontado por L . Assim, como a célula nova será inserida no início da lista, o membro “anterior” dessa célula recebe o valor *Null* (linha 4).

A linha 5 do algoritmo verifica se a lista não é vazia; caso de fato não seja, o membro “anterior” da célula apontada por L armazenará o endereço da nova célula. Caso a lista seja vazia, a variável L receberá o endereço da nova célula.

```
InserirDLigada( $L$ ,  $k$ )
1.   $nova \leftarrow$  endereço de uma nova célula
2.   $nova.valor \leftarrow k$ 
3.   $nova.posterior \leftarrow L$ 
4.   $nova.anterior \leftarrow Null$ 
5.  se  $L \neq Null$ 
6.       $L.anterior \leftarrow nova$ 
7.   $L \leftarrow nova$ 
```

A operação de remoção em uma lista duplamente ligada é feita a partir da identificação do endereço de memória da célula que se quer remover. Desse modo, a remoção segue de forma distinta daquela apresentada para uma lista ligada simples. Para esse caso, cujos passos são descritos no Algoritmo 3, existe a combinação da operação de busca e remo-

ção em um único algoritmo. Com o objetivo de apresentar a maior quantidade possível de configurações, para as operações sobre conjuntos dinâmicos, a operação de remoção em uma lista duplamente ligada é apresentada separadamente da operação de busca.

A Figura 6 ilustra a operação de remoção em uma lista duplamente ligada. A célula a ser removida é indicada pelo ponteiro q . Pode parecer estranho identificar o elemento a ser removido a partir de seu endereço em vez do seu valor, porém se pode utilizar o valor para encontrar o endereço por meio da utilização do Algoritmo 1.

A operação de remoção consiste em “conectar” as células apontadas por $q.anterior$ e $q.posterior$, de modo que a célula apontada por q seja removida da lista.

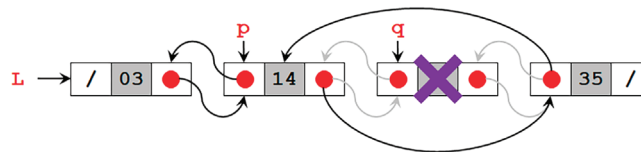


Figura 6 – Operação de remoção em uma lista duplamente ligada

Fonte: Acervo do Conteudista

Algoritmo 5 – Algoritmo para remoção em uma lista duplamente ligada

O primeiro passo para a operação de remoção é criar um ponteiro p e atribuir a ele o endereço do membro $q.anterior$; esse passo é descrito na linha 5 no Algoritmo 5. Em seguida, deve-se “conectar” as células vizinhas a q , para isso o membro $p.posterior$ receberá o endereço de $q.posterior$ e, caso a célula a ser removida não seja o final da lista, o membro $q.posterior.anterior$ receberá o endereço em p ; esses passos são descritos nas linhas 6-8 no Algoritmo 5. Veja que a notação considerada para identificar os membros das células pode ser utilizada em cascata, assim $q.posterior.anterior$ se refere à célula seguinte a q e ao membro “anterior” dessa célula.

Um caso particular ocorre quando o elemento a ser removido é o primeiro da lista. Nesse caso, o ponteiro L deve receber o endereço armazenado no membro $q.posterior$, que indica o segundo elemento da lista. Esse segundo elemento se tornará o primeiro com a remoção e deverá armazenar em seu membro “anterior” o valor *Null*. As instruções referentes a esse caso particular são descritas nas linhas 1-3 no Algoritmo 5.

RemoverDLigada(q)

1. **se** $q.anterior = \text{Null}$
2. $L \leftarrow q.posterior$
3. $q.posterior.anterior \leftarrow \text{Null}$
4. **senão**
5. $p \leftarrow q.anterior$
6. $p.posterior \leftarrow q.posterior$
7. **se** $q.posterior \neq \text{Null}$
8. $q.posterior.anterior \leftarrow p$

As listas ligadas apresentam outras variações possíveis. Podemos ter uma lista ligada circular, na qual o membro “ponteiro” da última célula contém o endereço da primeira. Além disso, podemos implementar uma lista duplamente ligada circular, a qual apresenta as mesmas características da lista ligada circular simples com a utilização de um segundo ponteiro para a célula anterior.

Na próxima seção, veremos a implementação de uma fila, considerando conjuntos dinâmicos.

Fila Dinâmica

Na primeira unidade desta disciplina, exploramos o conceito de fila, sobre uma estrutura de dados estática. Uma fila armazena elementos de acordo com uma regra de acesso específica. A sigla *FIFO* (*First In First Out*) define a regra de acesso para uma fila como o primeiro elemento que entra será o primeiro a sair da fila. Como em outras estruturas de dados, a fila suporta algumas operações para a manipulação de seus elementos. Nesta unidade, discutiremos os detalhes que são pertinentes à implementação da fila e suas operações no contexto dos conjuntos dinâmicos. Antes de iniciar o assunto, é recomendável a releitura da unidade de título “Estruturas de dados estáticas”, para que se tenha o melhor aproveitamento possível.



As regras de acesso para estruturas como a fila ou a pilha são as mesmas, independentemente de considerarem um arranjo estático ou uma lista dinâmica. A diferença está na forma de manipular as estruturas.

A primeira diferença entre as implementações de uma fila, considerando-se estruturas estáticas e dinâmicas, é o conceito de fila cheia. Em uma estrutura estática, o limite de armazenamento é dado pelo tamanho da fila, o qual foi definido previamente. A fila dinâmica não apresenta essa limitação, visto que sempre será possível alocar mais espaço de memória, em tempo de execução, para o armazenamento de um novo elemento. De fato, o limite para a fila dinâmica está restrito à disponibilidade de memória.

Uma fila dinâmica F pode ser definida como uma estrutura composta por dois ponteiros: *head* e *tail*, que indicam os endereços inicial e final da fila, respectivamente. Nesse contexto, uma fila dinâmica é vazia ($IsEmpty(F)$) quando o membro $F.head = Null$. Veja que a inicialização de uma fila dinâmica considera a alocação da estrutura e a atribuição do valor *Null* a ambos os membros.

A Figura 7 ilustra a estrutura de uma fila dinâmica implementada sobre uma lista. De acordo com a regra de acesso FIFO, a inserção de um novo elemento na fila deverá ser feita a partir do endereço no membro $F.head$. Nesse contexto, a operação de inserção ($Enqueue()$) é descrita no Algoritmo 6. Os parâmetros de entrada são a própria fila F e o valor a ser inserido x .

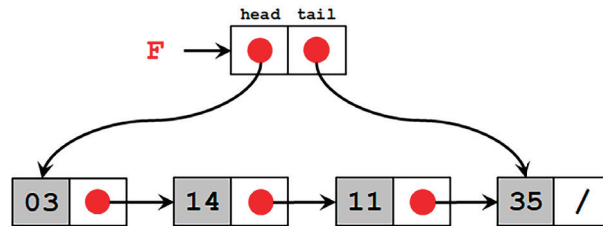


Figura 7 – Fila implementada sobre uma lista dinâmica

Fonte: Acervo do Conteudista

Algoritmo 6 – Algoritmo para inserção em uma fila dinâmica

Inicialmente, é alocada uma nova célula e seu endereço é atribuído à variável ponteiro *nova*. Os membros “valor” e “ponteiro”, na nova célula, recebem os valores a serem inseridos na fila e *Null*, essas atribuições são descritas nas linhas 1-3 no Algoritmo 6. Para o caso de a fila estar vazia, a célula *nova* será a única na fila; assim, o membro *F.head* receberá o endereço dessa nova célula (linhas 4-5). Caso a fila não esteja vazia, o endereço da nova célula será armazenado no membro “ponteiro” da última célula na fila, a qual é indicada pelo membro *F.tail*; veja as linhas 6-7 no Algoritmo 6. Finalmente, o final da fila é atualizado com o endereço na nova célula (linha 8).

Enqueue(F, x)

1. *nova* ← endereço de uma nova célula
2. *nova.valor* ← *x*
3. *nova.ponteiro* ← *Null*
4. **se** *IsEmpty(F)*
5. *F.head* ← *nova*
6. **senão**
7. *F.tail.ponteiro* ← *nova*
8. *F.tail* ← *nova*

A operação de remoção de um elemento em uma fila dinâmica considera a célula indicada pelo ponteiro *head*. Isso está de acordo com a regra de acesso às filas, que diz que as remoções devem ser feitas no início da fila. O Algoritmo 6 descreve os passos para a execução da operação *Dequeue()*.

A remoção somente poderá ser realizada mediante a existência de elementos na fila; assim, esta verificação é a primeira instrução do algoritmo (linha 1). Caso haja elementos na fila, o valor desse elemento é reservado, por meio da sua atribuição à variável *x* (linha 2).

Algoritmo 7 – Algoritmo para remoção em uma fila dinâmica

A instrução seguinte se refere à atualização do início da fila. O novo endereço de início da fila é aquele armazenado no membro “ponteiro” da célula que será removida. Essa atualização de ponteiros é descrita na linha 3 do Algoritmo 7. Após a atualização do início da fila, é verificado se ainda há elementos na estrutura; caso não haja, o en-

dereção no membro “Tail” é atualizado para refletir essa configuração. A verificação e a atualização correspondente são descritas nas linhas 4-5.

Finalmente, o algoritmo retornará o valor que foi removido da fila (linha 6), lembrando que esse valor foi reservado, previamente, na variável x , conforme descrito na linha 2.

```

Dequeue(F)
1.  se !IsEmpty(F)
2.     $x \leftarrow F.head.valor$ 
3.     $F.head \leftarrow F.head.ponteiro$ 
4.    se  $F.head = Null$ 
5.       $F.tail \leftarrow Null$ 
6.    retornar  $x$ 
7.  senão
8.    erro underflow
    
```

As operações de inserção e de remoção são ilustradas na Figura 8. No primeiro exemplo, representado em (a), o valor 21 está sendo inserido na fila dinâmica F . Veja que uma nova célula é alocada e recebe o valor e um ponteiro para $Null$. Em seguida, os ponteiros são atualizados com $F.tail.ponteiro$ e $F.tail$ recebendo a atribuição do endereço da nova célula.

Na Figura 8b, o valor 03 está sendo removido da fila, a partir da operação $Dequeue()$. A primeira etapa consiste em reservar o valor a ser removido na variável x . Em seguida, o endereço da próxima célula é atribuído ao membro $F.tail$, de modo a atualizar o início da fila após a remoção. O valor reservado em x será retornado para a chamada da função de remoção.

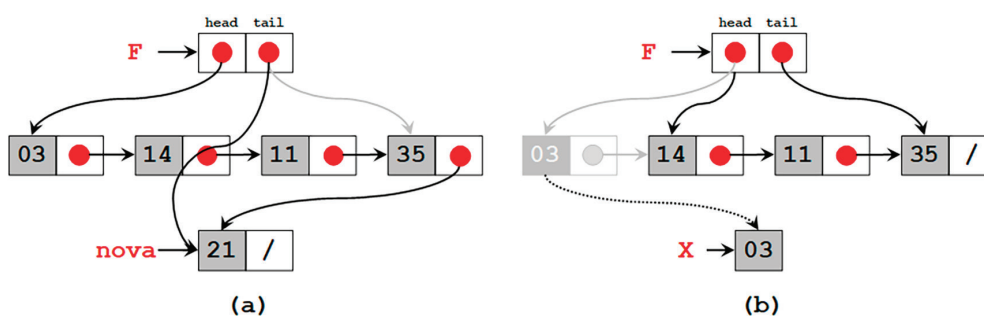


Figura 8 – Exemplos das operações de inserção e de remoção em filas dinâmicas
Fonte: Acervo do Conteudista

Pilha Dinâmica

Assim como a fila, a pilha é uma estrutura de dados que apresenta uma regra de acesso específica. A sigla *LIFO* (*Last In First Out*) descreve a regra de acesso à pilha como o último elemento que entra é o primeiro que sai. Os detalhes dessa estrutura foram descritos na unidade 1 desta disciplina, com a sua implementação na versão estática.

A Figura 9 ilustra as operações de empilhar (*Push()*) e de desempilhar (*Pop()*) elementos em uma pilha dinâmica. A versão utilizada na pilha dinâmica da figura considera uma “cabeça” para a estrutura, que é um ponteiro para uma célula, o qual não comporá a pilha. A chamada ao procedimento *Push(P,3)*, o qual é descrito no Algoritmo 8, aloca uma nova célula e atribui o valor de 3 ao membro *nova.valor*.

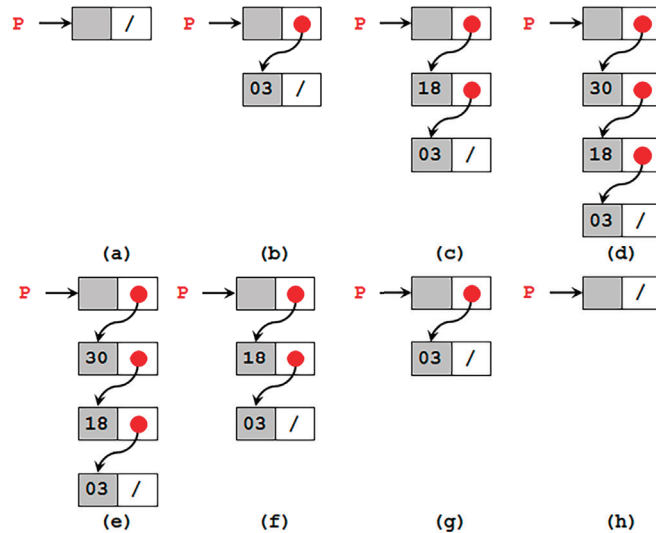


Figura 9 – Exemplos das operações de empilhar e de desempilhar em pilhas dinâmicas

Fonte: Acervo do Conteudista

Algoritmo 8 – Procedimento para inserir o valor na pilha dinâmica P

O membro *nova.ponteiro* recebe o valor armazenado no membro *P.ponteiro*, que, no caso, é igual a *Null*, por se tratar do primeiro elemento inserido. A atualização dos ponteiros é finalizada com a atribuição do endereço da célula *nova* ao membro *P.ponteiro*, de modo a “conectar” a “cabeça” da pilha ao primeiro elemento inserido. As Figuras 9a-d ilustram a inserção de elementos na pilha dinâmica, considerando as instruções descritas no Algoritmo 8.

Push(P, x)

1. *nova* $\leftarrow$ endereço de uma nova célula
2. *nova.valor* $\leftarrow$ *x*
3. *nova.ponteiro* $\leftarrow$ *P.ponteiro*
4. *P.ponteiro* $\leftarrow$ *nova*

A operação de desempilhar (*Pop()*) somente deve ser realizada a partir da existência de elementos na pilha. A função *IsEmpty()* realiza a verificação da existência de elementos na pilha por meio do retorno do valor lógico da expressão *P.ponteiro = Null*.

As Figuras 9e-h ilustram uma sequência de operações de desempilhar até que a pilha fique, novamente, vazia. O valor que será desempilhado é reservado na variável *x* para que seja retornado ao final da execução da função. Essa instrução está descrita na linha 2 do Algoritmo 9.

Algoritmo 9 – Função para remover o valor posicionado no da pilha V

Após a reserva do valor desempilhado, o ponteiro para o topo da pilha ($P.ponteiro$) é atualizado, passando a apontar para o segundo valor, indicando o novo topo (linha 3). Essa operação é ilustrada na Figura 9; veja que as chamadas se sucedem até que a pilha esteja vazia. O Algoritmo 9 prevê um retorno de erro quando a operação for realizada sobre uma pilha vazia.

```

Pop(P)
1.  se !IsEmpty( )
2.       $x \leftarrow P.ponteiro.valor$ 
3.       $P.ponteiro \leftarrow P.ponteiro.ponteiro$ 
4.      retorna  $x$ 
5.  senão
6.      erro underflow
    
```

Algoritmos de Classificação em Estruturas Dinâmicas

Os algoritmos de classificação, ou ordenação, foram discutidos nas unidades anteriores, considerando a implementação sobre conjuntos de dados estáticos (arranjos). Essa classe de algoritmos pode ser considerada, também, sobre estruturas dinâmicas. Nesta unidade, vamos estudar a versão dinâmica para o algoritmo de ordenação por inserção (*Insertion Sort*).

A ideia fundamental, por trás da versão dinâmica do *Insertion Sort*, é a remoção de um elemento da lista dinâmica inicial e inseri-lo em uma nova lista em sua posição correta. Assim, o algoritmo considera um ponteiro t que indica o início da lista e faz com que o ponteiro L indique uma lista vazia. Os elementos são removidos de e e inseridos, de forma ordenada, em L . A Figura 10a apresenta um exemplo de lista dinâmica, referenciada pelo ponteiro L .

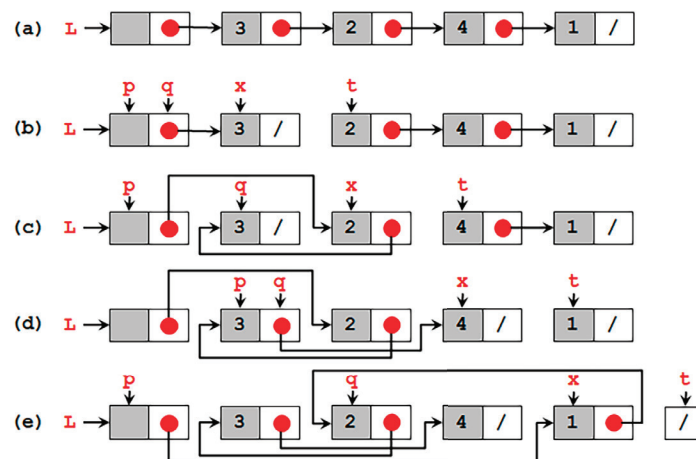


Figura 10 – Exemplo de ordenação (*Insertion Sort*) sobre uma lista dinâmica

Fonte: Acervo do Conteudista

Inicialmente, é criado um novo ponteiro que indicará o início da lista, ou seja, o endereço em L é copiado para o novo ponteiro; veja na linha 1 do Algoritmo 10 essa instrução. Em seguida, o ponteiro L é “desconectado” da lista inicial, assim ele receberá os elementos da lista de forma ordenada (linha 2).

O laço, descrito na linha 3 do Algoritmo 10, é responsável por percorrer os elementos da lista. O ponteiro x indica o elemento que será removido da lista inicial e inserido, em sua posição correta, na nova lista. O ponteiro é atualizado de modo a indicar o elemento posterior a x . O procedimento *Inserir()*, descrito no Algoritmo 11, recebe como parâmetros de entrada os ponteiros L e x ; veja as linhas 3-6 do Algoritmo 10.

Algoritmo 10 – Algoritmo Insertion Sort para ordenar uma lista dinâmica

O procedimento *Inserir()* cria dois novos ponteiros, p e q , que indicarão a posição inicial da nova lista e a posição de inserção do elemento x , respectivamente; veja as linhas 1-2 do Algoritmo 11. Um laço percorre a nova lista de modo a identificar a posição de inserção do elemento x ; nesse laço, os ponteiros p e q marcam a posição de inserção, comparando os valores nos membros $x.valor$ e $q.valor$ (linhas 3-5). Após a identificação da posição, a célula x é inserida na nova lista (linhas 9-10). A Figura 10b mostra o estado da nova lista após a primeira iteração do *Insertion Sort*.

InsertionSort(L)

1. $t \leftarrow L.ponteiro$
2. $L.ponteiro \leftarrow \text{Null}$
3. **enquanto** $t \neq \text{Null}$
4. $x \leftarrow t$
5. $t \leftarrow t.ponteiro$
6. $\text{Inserir}(L, x)$

Após a primeira iteração, o algoritmo *InsertionSort()* atualiza os ponteiros x e t , que definem o elemento que será retirado e o início da lista original, respectivamente. Assim, o elemento em x será inserido na nova lista, em sua posição correta, por meio do procedimento *Inserir()*.

Algoritmo 11 – Procedimento auxiliar para inserção ordenada em uma lista dinâmica

Veja na Figura 10c o estado da lista após uma nova iteração. Note que a lista apontada por L possui dois elementos ordenados (2 e 3) e a lista inicial, apontada por t , possui os elementos remanescentes desordenados (1 e 4). Se considerarmos a disposição desses valores na memória, é possível observar que suas localizações não se alteraram, somente os ponteiros em cada célula são atualizados de modo a sequenciar os elementos corretamente.

Inserir(L, x)

1. $p \leftarrow L$
2. $q \leftarrow L.ponteiro$
3. **enquanto** $q \neq \text{Null}$ e $q.valor < x.valor$

4. $p \leftarrow q$
5. $q \leftarrow q.ponteiro$
6. $x.ponteiro \leftarrow q$
7. $p.ponteiro \leftarrow x$

O algoritmo *InsertionSort()* finaliza quando todos os elementos são retirados da lista inicial e inseridos, na posição correta, na nova lista. A Figura 10e ilustra o estado final da lista após a finalização da ordenação. Note que o resultado é a atualização dos ponteiros nos membros de cada célula, sem de fato alterar a localização das células na memória.

Pesquisa em Estruturas Dinâmicas

Os métodos de pesquisa ou busca, estudados anteriormente, consideraram uma estrutura de dados estática, mais precisamente um arranjo. Nesse contexto, a pesquisa sequencial, discutida na unidade de título “Algoritmos de Classificação e Pesquisa I”, é realizada percorrendo todo o arranjo e comparando cada elemento com aquele que é procurado. Essa abordagem é simples de ser implementada, porém pode não ser eficiente do ponto de vista do tempo que se gasta para a identificação de um elemento em um arranjo com muitos elementos.

Realizar a pesquisa sequencial em uma estrutura dinâmica, como as listas apresentadas anteriormente, segue de forma similar à abordagem estática. O Algoritmo 1 descreve os passos para a realização de busca em uma lista ligada. Veja que não há grandes diferenças ao compararmos com a busca sobre um arranjo (unidade “Algoritmos de Classificação e Pesquisa I”).

Outra abordagem, discutida na unidade de título “Algoritmos de Classificação e Pesquisa II”, é a pesquisa binária. Vimos que esse tipo de pesquisa requer um arranjo ordenado. Assim, a partir da identificação da posição média no arranjo podemos identificar a qual das partições, resultantes a partir do ponto médio, pertence o elemento buscado. Desse modo, é possível reduzir significativamente o número de operações até encontrar o valor de interesse.

A busca binária realizada sobre uma lista ligada é mais problemática comparada àquela realizada sobre um arranjo estático. O grande problema aqui está em identificar o ponto médio da lista ligada. Veja que não é uma tarefa difícil, porém a identificação do ponto médio em uma lista ligada depende da realização do percurso por todos os elementos na lista. Nesse caso, o número de operações que serão realizadas pela pesquisa binária é maior que o número de operações realizadas pela pesquisa sequencial.

Existem abordagens que buscam otimizar a realização da pesquisa binária sobre uma lista encadeada. Porém, a abordagem da pesquisa binária parece de fato incompatível com as características de uma lista ligada. Veja que a ausência de linearidade nos endereços armazenados pelos ponteiros dificulta, de maneira importante, a realização da busca binária sem prejuízo ao ganho de desempenho observado nas estruturas estáticas.

Nesta unidade, discutimos a aplicação de alguns conceitos anteriores sobre estruturas de dados dinâmicas. Assim, tivemos a oportunidade de observar as diferentes características dos algoritmos considerando as estruturas estáticas e as dinâmicas. Na próxima unidade, voltaremos ao estudo da classe de algoritmos de ordenação, com foco em algoritmos rápidos. Além disso, teremos um contato inicial com a área de análise de algoritmos, que se dedica ao estudo do desempenho dos algoritmos quanto ao tempo e espaço consumidos para sua execução.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Vídeos

Listas dinâmicas encadeadas em C – Estrutura de dados

<https://youtu.be/Fc83QKzs9xU>

Estrutura de Dados – Aula 5

<https://youtu.be/OE3CtV2bGqo>

Estrutura de Dados – Aula 6

<https://youtu.be/C6WOW0L1X04>

Leitura

Implementation of binary search on a singly linked list using dual pointers

<https://bit.ly/315wsXL>

Referências

ASCENCIO, A. F. G.; ARAÚJO, G. S. **Estruturas de dados**: algoritmos, análise da complexidade e implementações em *Java* e *C/C++*. São Paulo: Pearson, 2010. (*e-book*)

CORMEN, T. H. *et al.* **Introduction to algorithms**. MIT press, 2009.

EDELWEISS, N. **Estruturas de dados**. Porto Alegre: Bookman, 2011. (*e-book*)

SZWARCFITER, J. L. **Estruturas de dados e seus algoritmos**. 3^a ed. Rio de Janeiro: LTC, 2010. (*e-book*)



Cruzeiro do Sul
Educatonal